SPAWNTECH

SpawnTech Nexus

Arquitectura distribuida para la gestión de solicitudes de soporte técnico y desarrollo de software

Actividad: 4. Tejiendo redes: Arquitectura de software entre hilos y nodos

Curso: Arquitectura de Software — Unidad 2

Institución: [Completar nombre de la institución]

Estudiante: [Completar nombre completo]

Rol(es): líder, arquitecto, desarrollador, probador y expositor (trabajo individual)

Docente: [Completar nombre del docente]

Fecha de elaboración: 9 de noviembre de 2026

Fecha de entrega: [Completar según LMS]

Nota académica: la guía pide un equipo de 4 a 5 integrantes; este proyecto se elaboró para un único participante y requiere aval del docente para presentarse de forma individual.

Índice

Índice 2

1. Introducción 3

2. Objetivos 3

- 2.1 Objetivo general . . . 3
- 2.2 Objetivos específicos . . . 3

3. Descripción del sistema distribuido 3

- 3.1 Supuestos y delimitación . . . 4

4. Análisis y planificación 4

- 4.1 Requisitos . . . 4
- 4.2 Hilos, nodos y comunicación . . . 4
- 4.3 Roles y cronograma de referencia . . . 4

5. Modelado UML 5

- 5.1 Diagrama de componentes . . . 5
- 5.2 Diagrama de secuencia . . . 5
- 5.3 Diagrama de despliegue . . . 6

6. Selección y justificación de patrones 7

7. Implementación del prototipo 7

- 7.1 Estructura y tecnologías . . . 7
- 7.2 Procedimiento de ejecución . . . 7
- 7.3 Concurrencia y fallos . . . 7

8. Evaluación del desempeño y ajustes 8

- 8.1 Método y escenarios . . . 8
- 8.2 Pruebas de robustez . . . 8
- 8.3 Cuellos de botella y mejoras . . . 9
- 8.4 Amenazas a la validez . . . 9

9. Riesgos, ética y proyección 9

10. Conclusiones 9

11. Referencias bibliográficas (APA 7) 9

12. Anexos y enlaces de entrega 10

Enlaces pendientes de la entrega: Repositorio GitHub/GitLab: [INSERTAR URL]. Video de YouTube no listado: [INSERTAR URL].

Estado de evidencia: el prototipo y sus pruebas sí fueron ejecutados en localhost. No se ha creado automáticamente un repositorio remoto ni se ha grabado o publicado un video.

1. Introducción

SpawnTech es un emprendimiento tecnológico enfocado en el desarrollo de software y el soporte técnico. A medida que aumentan las solicitudes de clientes, gestionarlas desde un único componente puede crear demoras, dificultar la trazabilidad y generar un punto único de fallo. Este mini proyecto propone una arquitectura distribuida lógica, de alcance académico, para recibir, clasificar y registrar tales solicitudes mediante servicios HTTP independientes.

Se presenta un prototipo llamado SpawnTech Nexus. El sistema no realiza reparaciones ni construye software de manera automática; clasifica cada petición y la registra en una cola lógica de atención. Se estudian las relaciones entre procesos, hilos y nodos, los patrones de diseño y los efectos de habilitar uno o dos procesadores concurrentes.

2. Objetivos

2.1 Objetivo general

Diseñar, modelar, implementar y evaluar un prototipo de arquitectura distribuida para la gestión de solicitudes de SpawnTech, aplicando patrones de diseño, diagramas UML y pruebas reproducibles de concurrencia, escalabilidad y tolerancia a fallos.

2.2 Objetivos específicos

- Identificar requisitos funcionales y no funcionales relacionados con atención, concurrencia, trazabilidad y disponibilidad.
- Modelar la distribución funcional, las interacciones temporales y el despliegue mediante UML de componentes, secuencia y despliegue.
- Aplicar Proxy, Mediator y una implementación simplificada de Observer, complementados por reintento entre nodos.
- Ejecutar pruebas de integración, cargas comparativas y fallos inducidos; documentar resultados y limitaciones.

3. Descripción del sistema distribuido

El cliente registra una solicitud de soporte o de desarrollo en el API Gateway. El Gateway valida los datos, asigna un identificador único y selecciona un nodo procesador mediante round-robin. Cada procesador simula 120 ms de trabajo de clasificación, limitado a dos tareas activas simultáneas mediante un semáforo. El Gateway almacena el resultado en SQLite e informa a un observador HTTP.

Cliente HTTP	Envía POST /tickets; consulta GET /tickets, /tickets/{id} y /stats.
API Gateway / coordinador	Proxy de acceso, Mediator de nodos, validación, reintento y almacenamiento.
Procesador A y B	Servicios HTTP independientes, múltiples hilos de atención y capacidad activa delimitada.
Observador	Recibe POST /events y consulta GET /events (memoria no durable).

SQLite	Repositorio local de tickets gestionado solo por el Gateway.

3.1 Supuestos y delimitación

- La red usada para la simulación es localhost (127.0.0.1), por lo que los nodos son procesos lógicos en una sola máquina; no es despliegue multinube ni red física multipunto.
- El esfuerzo del nodo es una demora controlada y simula trabajo I/O; no representa un diagnóstico real de equipos.
- Python 3.10+ y su biblioteca estándar bastan para ejecutar y probar el sistema. Los diagramas se entregan también en formato PlantUML editable.
- La persistencia es centralizada en SQLite; el observador guarda eventos en memoria, susceptible a pérdidas al reiniciar.

4. Análisis y planificación

4.1 Requisitos

RF-01	Funcional	Registrar una solicitud con título, categoría y descripción opcional; entregar UUID.
RF-02	Funcional	Enrutar solicitudes a nodos de atención según round-robin y categoría.
RF-03	Funcional	Consultar solicitudes persistidas y estadísticas por nodo.
RF-04	Funcional	Emitir un evento de ticket creado hacia el observador.
RNF-01	Calidad	Atender clientes simultáneos y limitar a 2 trabajos activos por nodo.
RNF-02	Calidad	Ante caída de un nodo, intentar procesar la solicitud en otro disponible.
RNF-03	Calidad	Demostrar y medir diferencia entre 1 y 2 nodos con la misma carga.

4.2 Hilos, nodos y comunicación

Cada servicio usa ThreadingHTTPServer, que crea hilos para manejar solicitudes HTTP. En cada nodo procesador, un semáforo restringe a dos el número de tareas de procesamiento simultáneas, aunque puedan existir más hilos esperando. Las llamadas HTTP representan comunicación entre procesos a través del stack de red local. La elección de round-robin y el acceso al repositorio se protegen con threading.Lock.

4.3 Roles y cronograma de referencia

Por tratarse de un emprendimiento operado por una sola persona, el mismo estudiante asume las funciones de dirección, diseño UML, implementación Python, evaluación, documentación y exposición. La autorización académica del trabajo individual queda pendiente de confirmación.

Semana 1	Requisitos y alcance; diseño de UML.	Líder + arquitecto
Semana 2	Implementación de Gateway, nodos y observador.	Desarrollador
Semana 3	Pruebas, fallo inducido, comparación y ajustes.	Evaluador de calidad
Semana 4	Informe PDF, exposición y repositorio.	Documentador + expositor

5. Modelado UML

Se emplean los diagramas de componentes, secuencia y despliegue de UML 2.5.1 (Object Management Group, 2017). Los fuentes editables .puml se incluyen en la carpeta arquitectura/.

5.1 Diagrama de componentes

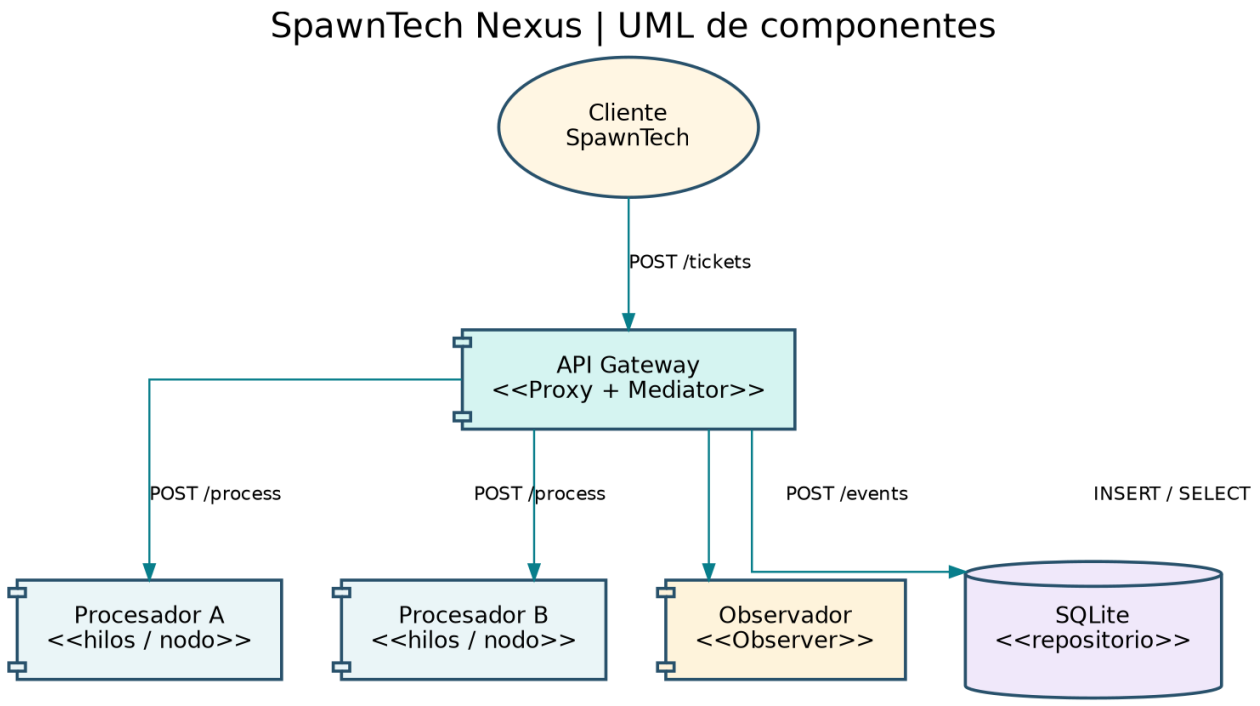


Figura 1. Componentes lógicos, contratos HTTP y almacenamiento. Fuente: elaboración propia.

La separación de componentes permite estudiar la responsabilidad única de cada módulo. El Gateway centraliza el acceso y controla a qué procesador delega; el observador recibe una notificación independiente del resultado de clasificación. El repositorio central simplifica este prototipo, aunque mantiene una dependencia crítica.

5.2 Diagrama de secuencia

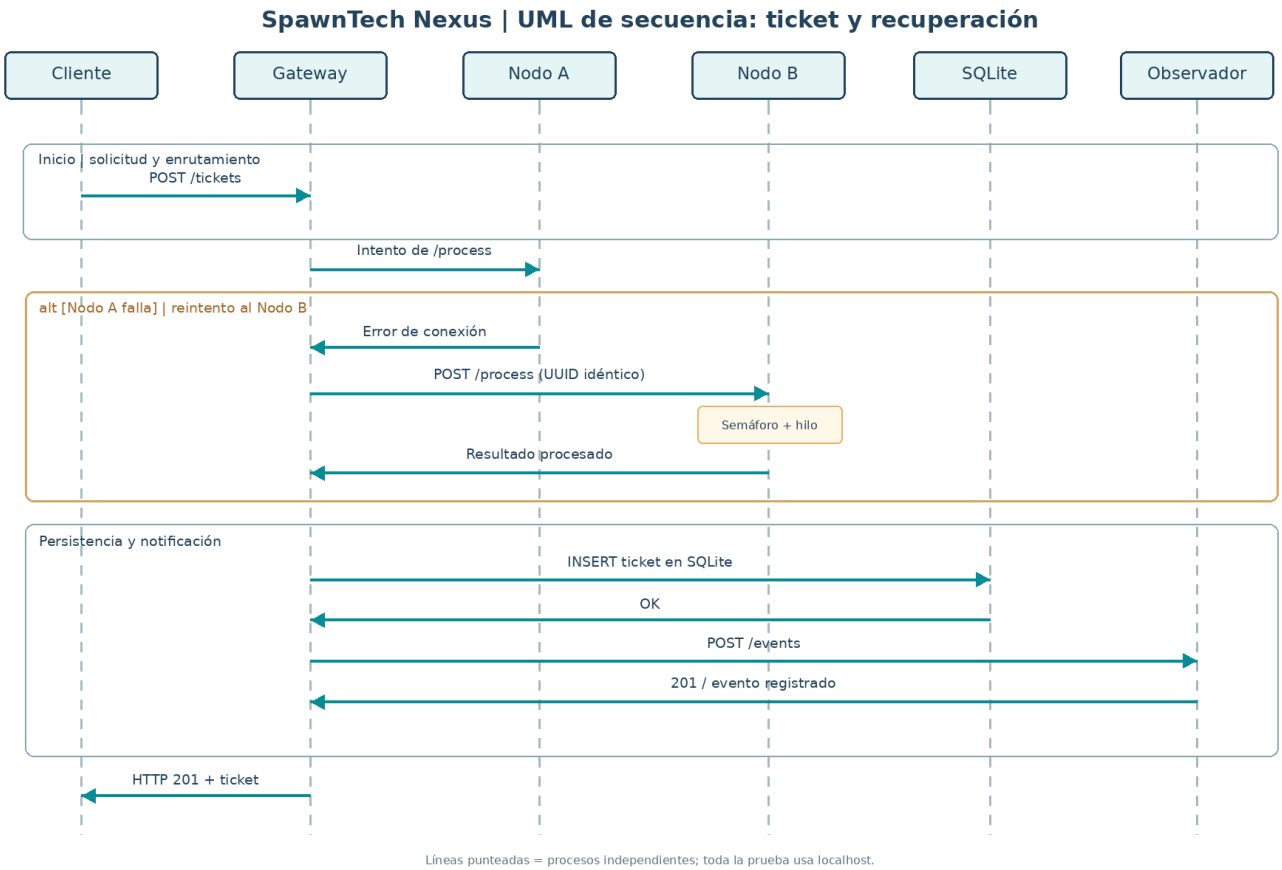


Figura 2. Flujo de un ticket, reintento, persistencia y notificación. Fuente: elaboración propia.

La secuencia muestra el camino de recuperación ante un fallo del nodo A. El Gateway reutiliza el mismo UUID en el reintento hacia B y confirma la solicitud al cliente después de obtener respuesta del procesador y guardar el ticket. El envío al observador es de mejor esfuerzo: un fallo allí aumenta un contador, pero no anula la solicitud.

5.3 Diagrama de despliegue

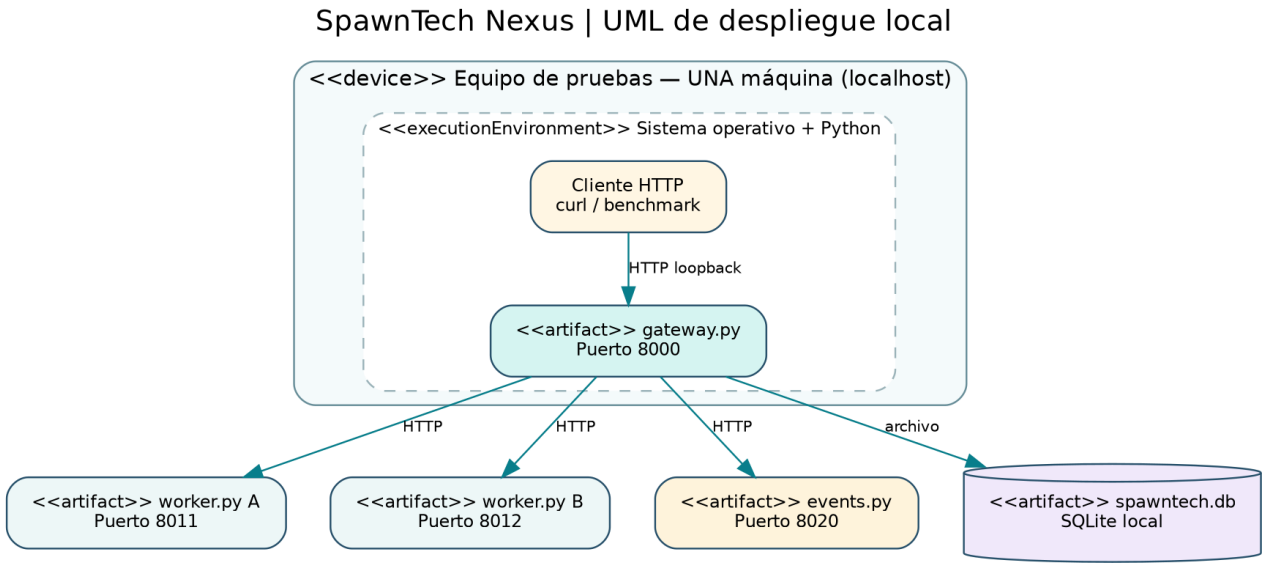


Figura 3. Despliegue físico-lógico en un equipo de pruebas. Fuente: elaboración propia.

El despliegue muestra procesos aislados por puerto en el mismo sistema operativo. Para convertirlo en un sistema distribuido físicamente sería necesario distribuir procesos en hosts o contenedores de red distintos,

configurar descubrimiento, autenticar conexiones y usar almacenamiento accesible y resiliente.

6. Selección y justificación de patrones

Proxy	La API es el punto de acceso externo a los procesadores.	Oculta direcciones internas y valida; agrega un salto HTTP.
Mediator	El Gateway decide y coordina nodos, almacena y notifica.	Reduce acoplamiento cliente/nodo; concentra funciones y riesgo.
Observer (simplificado)	El observador de eventos es notificado mediante POST /events.	Separa seguimiento y atención; sin broker ni entregas garantizadas.
Retry / failover	Ante excepción de conexión, Gateway intenta otro nodo.	Mejora disponibilidad parcial; tiempo adicional y semántica de reintento.

La elección retoma los principios de patrones de diseño (Gamma et al., 1994), la semántica de reintentos y fallos distribuidos descrita en Azure Architecture Center (Microsoft, s. f.-a, s. f.-b) y el desacoplamiento por eventos (Microsoft, s. f.-c). La solución no incluye un circuit breaker completo ni un broker de mensajería, por lo que no se atribuyen esas capacidades al código.

7. Implementación del prototipo

7.1 Estructura y tecnologías

El repositorio listo para publicar contiene gateway.py, worker.py, events.py, common.py, cluster.py, launcher.py, benchmark.py, pruebas/test_integracion.py y los archivos de diagramación. Se utilizaron Python, SQLite, HTTP/JSON, hilos de servidor y semáforos. No existen dependencias de terceros para la ejecución.

7.2 Procedimiento de ejecución

Paso 1. Abrir una terminal en el directorio y ejecutar python launcher.py --workers 2. Se exponen los servicios de forma local: Gateway en el puerto 8000, nodos en 8011/8012 y observador en 8020.

Paso 2. En otra terminal, enviar un POST a http://127.0.0.1:8000/tickets con JSON: {"titulo":"Mi PC no inicia","categoria":"soporte","descripcion":"Solicito diagnóstico"}. Repetir con categoría "desarrollo".

Paso 3. Consultar GET /tickets, GET /stats y GET http://127.0.0.1:8020/events. Verificar el nodo que atendió, la cola lógica y la notificación.

Paso 4. Detener la demostración con Ctrl+C; ejecutar python -m unittest discover -s pruebas -v y python benchmark.py. El segundo comando genera un JSON nuevo de métricas locales.

7.3 Concurrencia y fallos

El procesador utiliza hilos para servir conexiones y un BoundedSemaphore(2) para limitar dos tareas activas. El Gateway combina un índice round-robin protegido por mutex y reintentos secuenciales hacia los nodos configurados. Los identificadores UUID se conservan durante el reintento. El observador opera en otro proceso y su indisponibilidad se refleja en la métrica fallos_observador.

8. Evaluación del desempeño y ajustes

8.1 Método y escenarios

Se ejecutaron pruebas automáticas el 9 de noviembre de 2026 en Python 3.13.5 sobre Linux, usando localhost. La prueba de carga incluyó 48 peticiones con 12 clientes simultáneos, una pasada por escenario, después de cuatro solicitudes de calentamiento. El trabajo simulado por procesador fue de 120 ms, con un máximo de dos trabajos activos por nodo. Todas las cifras a continuación corresponden a esta ejecución y pueden variar en otros equipos.

Solicitudes exitosas	48/48	48/48
Tiempo total	2.920 s	1.483 s
Productividad	16.44 sol./s	32.36 sol./s
Latencia p50	722.34 ms	360.84 ms
Latencia p95	732.39 ms	378.53 ms
Distribución de atención	A: 48	A: 24; B: 24

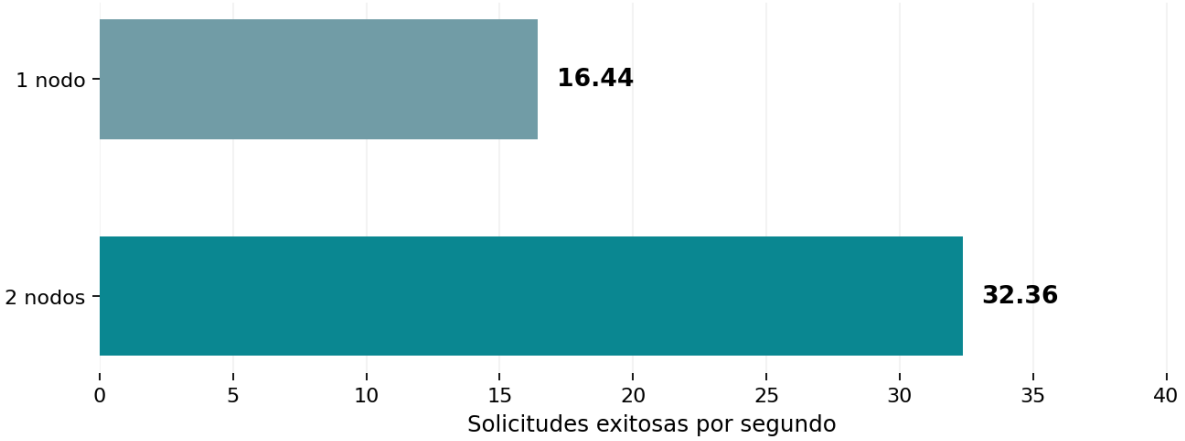


Figura 4. Productividad observada con la misma carga en localhost. Datos: pruebas/resultados_reales.json.

La productividad aumentó un 96.8 % al pasar de uno a dos nodos en este ensayo, y se redujo la latencia p50. El beneficio es coherente con la capacidad paralela disponible (dos tareas por nodo). No debe interpretarse como una garantía de aumento lineal bajo cargas reales o redes externas.

8.2 Pruebas de robustez

- Caída del nodo A: se detuvo su proceso. Se completaron 12/12 solicitudes redirigidas al nodo B; no hubo errores en esta prueba.
- Caída del observador: se detuvo su proceso antes de una nueva petición. El ticket se creó y persistió; el Gateway contabilizó un fallo de notificación, cuyo evento no se conserva de forma durable.
- Pruebas de integración: cuatro casos pasaron: clasificación de categorías y persistencia, recepción de eventos, validación de datos erróneos (HTTP 400), y uso de ambos nodos por round-robin.

8.3 Cuellos de botella y mejoras

El principal cuello de botella observado fue el límite de dos trabajos activos por nodo, visible en las latencias de espera. El ajuste implementado consistió en habilitar un segundo proceso procesador, conservar la asignación round-robin y proporcionar reintento ante caída del primero. Como mejoras futuras: base de datos externa, cola durable para notificaciones, observabilidad, autenticación, límites de tasa, balanceo por capacidad y despliegue en máquinas diferentes.

8.4 Amenazas a la validez

El experimento usa un solo equipo y no incorpora latencia de red real, fallas de partición, cifrado, acceso multiusuario ni cargas variables. Solo se ejecutó una pasada por configuración; no se realizaron intervalos de confianza. El código didáctico de http.server carece de endurecimiento de seguridad para uso productivo (Python Software Foundation, s. f.).

9. Riesgos, ética y proyección

El prototipo no recoge datos reales de clientes. Para aplicaciones comerciales deben implementarse políticas de privacidad, credenciales, control de acceso por rol, TLS y separación entre ambientes. También se requiere atender la protección de datos personales y las obligaciones contractuales pertinentes antes de registrar casos reales de soporte o desarrollo.

El crecimiento de SpawnTech puede comenzar con la arquitectura conceptual propuesta, pero la adopción productiva exige migración gradual a un servidor HTTP de producción, autenticación, base central confiable, mensajería durable, logs con ID de correlación y pruebas de desastre en redes físicamente distribuidas.

10. Conclusiones

- Se diseñó una arquitectura distribuida lógica pertinente para SpawnTech, que separa la entrada de solicitudes, el trabajo de clasificación y la recepción de eventos.
- Los diagramas UML permiten argumentar distribución de responsabilidades, comunicaciones y escenarios de recuperación.
- Proxy, Mediator y Observer simplificado aportan una organización entendible y extensible; el Gateway permanece como riesgo de punto único de fallo.
- Las pruebas locales mostraron una mejora de productividad del 96.8 % al habilitar un segundo procesador y recuperación satisfactoria ante la caída de uno de los nodos.
- Los resultados son académicos y medidos, no validan un producto comercial; la evolución requiere infraestructura real, seguridad y pruebas rigurosas.

11. Referencias bibliográficas (APA 7)

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design patterns: Elements of reusable object-oriented software. Addison-Wesley.

Microsoft. (s. f.-a). Circuit Breaker pattern. Azure Architecture Center. https://learn.microsoft.com/en-us/azure/architecture/patterns/circuit-breaker

Microsoft. (s. f.-b). Retry pattern. Azure Architecture Center. https://learn.microsoft.com/en-us/azure/architecture/patterns/retry

Microsoft. (s. f.-c). Event-driven architecture style. Azure Architecture Center. https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/event-driven

Object Management Group. (2017). OMG Unified Modeling Language (OMG UML), version 2.5.1. https://www.omg.org/spec/UML/2.5.1

Python Software Foundation. (s. f.). http.server — HTTP servers. Python documentation. https://docs.python.org/3/library/http.server.html

Python Software Foundation. (s. f.). threading — Thread-based parallelism. Python documentation. https://docs.python.org/3/library/threading.html

12. Anexos y enlaces de entrega

Anexo A. Fuentes UML: arquitectura/componentes.puml, arquitectura/secuencia.puml y arquitectura/despliegue.puml; representaciones PNG en imagenes/.

Anexo B. Código: incluir los archivos Python y pruebas en un repositorio propio. URL del repositorio: [PEGAR ENLACE GITHUB/GITLAB AQUÍ].

Anexo C. Evidencia: pruebas/resultados_reales.json, generado automáticamente por benchmark.py; comparación de 1 y 2 nodos y pruebas de fallo.

Anexo D. Presentación: archivo SpawnTech_Nexus_Presentacion.pptx y guion de 12–15 minutos en docs/guion_video.md. URL de video no listado: [PEGAR ENLACE AQUÍ].

Revisión previa a LMS: completar institución, nombre, docente, fecha y enlaces; confirmar autorización individual; grabar el video; subir archivos al repositorio y al aula virtual.